Analizador Semántico TS

Rocío Corral[1]

[1]Lic. Cs. de la Computación/Facultad de Ingeniería/UNCuyo

04 de mayo de 2023

Abstract

Decisiones de diseño, metodología, lenguaje utilizado, arquitectura, funcionamiento y resultados del proceso de implementación de un compilador para el lenguaje de programación TinyRust+ se explican en el presente trabajo.

1. Introducción

El objetivo de este informe es explicar y fundamentar la implementación de las primeras tres etapas del compilador de TinyRust+.

En la primera etapa se implementó el analizador léxico, programa que genera un conjunto de *tokens* a partir de un archivo de texto. Estos tokens se definen utilizando como base las reglas descritas en el manual de TinyRust+ para delimitar palabras reservadas, operadores, identificadores, tipos y demás.

En la segunda etapa se implementó el analizador sintáctico, programa que consume *tokens* generados por el analizador léxico y verifica que el programa cumpla con las reglas de la gramática de TinyRust+. Esta implementación consiste en un analizador sintáctico descendente predictivo recursivo. Para su implementación, se tuvo que tratar la gramática de TinyRust+ descrita en el manual transformándola a notación BNF, eliminando la recursividad a izquierda y factorizando.

En la tercera etapa se implementó la tabla de símbolos y el análisis semántico de declaraciones. La tabla de símbolos se construye de forma intercalada a medida que se realiza el análisis sintáctico del archivo de entrada. Durante el análisis sintáctico se agregan entidades a la tabla y para cuando se ha completado el análisis exitosamente, la tabla de símbolos está completa y se realiza su consolidación.

2. Metodología

Para el desarrollo de este proyecto, el primer paso fue hacer un análisis de la información disponible sobre el funcionamiento esperado de TinyRust+ y definir una serie de aspectos de diseño detallados en la siguiente sección. Posterior a esto, se diagramó la arquitectura del proyecto y finalmente se implementó secuencialmente cada etapa.

A la hora de la implementación, se trabajó con el lenguaje de programación Java. Se utilizó Git junto con GitHub como herramienta de control de versiones, de manera tal que ambos integrantes del equipo pudieran trabajar colaborativa e independientemente en el

proyecto de manera simultánea.

2.1 Consideraciones

A continuación se listan una serie de consideraciones y decisiones de diseño relevantes antes de profundizar en la implementación del compilador.

El alfabeto de entrada del compilador consta de los caracteres con código ASCII desde 32 hasta 126[1]. Adicionalmente, también se incluyen los caracteres:

- Salto de línea (ASCII 10)
- Tabulación (ASCII 9)
- Fin de texto (ASCII 3)
- Tabulación vertical (ASCII 11)

En la Tabla 1 se encuentra definido por extensión el alfabeto de TinyRust+.

[space]	!	"	#	$	%	&	'	(	)	*	+	,	-	.	0	1	2	3	4
[tab]	5	6	7	8	9	:	;	<	=	>	?	@	A	B	C	D	E	F	G
[vtab]	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
[break]	[	]	_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
[etx]	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	\	/	^	

Tabla 1. Alfabeto de entrada de TinyRust+.

Cabe destacar que ciertos caracteres exclusivos del español, como “ñ” y vocales con tilde (tanto mayúsculas como minúsculas) no forman parte del alfabeto de entrada de este compilador.

El analizador léxico reconoce distintos tipos de tokens. Existe un token distinto para cada palabra reservada de TinyRust+, cada operador, cada tipo de literal, comentario, símbolo delimitador de la sintaxis y cada tipo de identificador. Posteriormente estos tokens son consumidos por el analizador sintáctico y en base a su tipo y lexema se determina el significado a nivel estructural del código de entrada.

2.2 Errores

En la etapa de diseño se previó que una serie de errores distintos pueden aparecer durante cada etapa del análisis. Por este motivo, los tipos de errores están clasificados según la etapa de análisis en la que aparezcan.

Los posibles errores previstos en la etapa del análisis léxico son:

- *Identificador mal formado*: error generado al encontrar un lexema que es cercano a un identificador pero no cumple con su sintaxis por algún motivo. Por ejemplo, la cadena ‘123var’ comienza con dígitos seguidos de letras, por lo cual no es ni un literal entero

ni un identificador. Otro ejemplo podría ser una cadena que comienza con minúsculas y, sin un espacio de por medio, tiene un "@", por ejemplo, "vari@ble".

- *Comentario sin cerrar*: error generado al encontrar una secuencia de apertura de comentario multilínea y luego llegar al final del archivo sin encontrar la secuencia de cierre, es decir, "*/".
- *Símbolo no perteneciente al alfabeto*: error generado al leer en el archivo de entrada un caracter que no esté incluido en el alfabeto Σ, como por ejemplo: "¬", "ñ", "á", etc.
- *Token desconocido*: error generado al encontrar un lexema que no cumple ningún patrón reconocible, por ejemplo, "|". Cabe destacar que "||" sí es reconocido como el operador lógico OR. Sin embargo, "|" por sí solo, no es un token de este compilador.
- *Literal inválido*: error generado al hallar literales de tipo cadena o caracter con una estructura inválida. Por ejemplo, literal cadena sin cerrar o literal caracter conteniendo más de un caracter.

Para la etapa de análisis sintáctico se previó un único error *Token inesperado*, generado cuando se encuentra un token que no coincide con ninguna regla gramatical de TinyRust+, teniendo en cuenta la secuencia previa de tokens del archivo.

En la etapa de análisis semántico de declaraciones se previó que pueden aparecer distintos errores de chequeo semántico durante la etapa de análisis propiamente y durante la posterior etapa de consolidación. Cada tipo de error se lista a continuación:

- *Constructor mal formado*: error generado al encontrar una declaración de un método de clase cuyo identificador es "create". El error sugiere la sintaxis apropiada para declarar el constructor, ya que es ligeramente diferente de la sintaxis normal de declaración de métodos.
- *Múltiples constructores*: error generado al encontrar dos constructores declarados en una misma clase, ya que el lenguaje soporta un único constructor por clase.
- *Entidad duplicada*: error generado al declarar dos clases o dos métodos de una misma clase o dos argumentos de un mismo método con el mismo identificador.
- *Herencia base ilegal*: error generado al hallar una clase que herede de cualquiera de las clases predefinidas de TinyRust+, estas son: IO, Object, Str, Char, I32, Bool y Array.
- *Error de consolidación*: todo error generado durante la etapa de consolidación se cataloga en este grupo. El error incluye información explicativa para el usuario sobre el escenario en que ocurrió. Los escenarios posibles de este error son: se redefinió un método estático o atributo de una superclase, se redefinió un método de una superclase con distinta firma, se encontró herencia circular o herencia de una clase no declarada.

2.3 Transformación de la gramática

La cátedra facilitó en el manual de TinyRust+ una versión de su gramática en formato EBNF. Para hacer uso de la gramática para la implementación del analizador sintáctico descendente predictivo recursivo, se aplicaron una serie de algoritmos para transformar la gramática, sin alterar el conjunto de cadenas generados por la gramática. Los algoritmos aplicados fueron: transformación a formato BNF, eliminación de reglas

lambda, eliminación de recursividad a izquierda y factorización. Finalmente, se calcularon el conjunto de primeros y siguientes para los símbolos no terminales de la gramática.

Para la implementación del analizador semántico de declaración se extendió con atributos y acciones semánticas la gramática previamente tratada. A continuación se encuentra la gramática aumentada.

Como referencia, las acciones semánticas de color verde crean o agregan entidades a la tabla de símbolos, mientras que las anotaciones azules unicamente sintetizan o establecen atributos.

```
Start ::= Clases Main
Clases ::= Clase Clases | λ

Main ::=  "fn" "main" "(" ")" Bloque-Método {ts.addMain()}

Clase ::= "class" "idClase" ClaseHerenciaOp {currClase = ts.addClase(idClase.lex)}

Herencia ::= ":" "idClase" {currClase.heredaDe = idClase.lex}

ClaseHerenciaOp ::= Herencia "{" Miembros "}"
    | "{" Miembros "}"
Miembros ::= Miembro Miembros | λ
Miembro ::= Atributo
    | Constructor
    | Método

Atributo ::= Tipo ":" Lista-Declaración-Variables ";" {LDV.tipo = Tipo.tipo}
    | Visibilidad Tipo ":" Lista-Declaración-Variables ";" {LDV.tipo = Tipo.tipo; LDV.public =
Visibilidad.public}

Constructor ::= "create" Argumentos-Formales Bloque-Método {currMetodo =
ts.addConstructorClase(currClase)}
Método ::= "fn" "idMétodoVariable" Argumentos-Formales "->" Tipo-Método Bloque-Método
{currMetodo = ts.addMetodoClase(currClase, new Metodo(nombre: idMetodoVariable.lex,
retornoTipo: TipoMetodo.tipo)}
    | Forma-Método "fn" "idMétodoVariable" Argumentos-Formales "->" Tipo-Método
Bloque-Método {currMetodo = ts.addMetodoClase(currClase, new Metodo(nombre:
idMetodoVariable.lex, retornoTipo: TipoMetodo.tipo, static: FormaMetodo.static)}

Argumentos-Formales ::= "(" ")"
    | "(" Lista-Argumentos-Formales ")"
Lista-Argumentos-Formales ::= Argumento-Formal "," Lista-Argumentos-Formales
    | Argumento-Formal

Argumento-Formal ::= Tipo ":" "idMétodoVariable" {currMetodo.addArgumento(nombre:
idMetodoVariable.lex, tipo: Tipo.tipo)}

Forma-Método ::= "static" {FormaMetodo.static = true}

Visibilidad ::= "pub" {Visibilidad.public = true}
```

```
Tipo-Método ::= Tipo {TipoMetodo.tipo = Tipo.tipo}
    | "void" {TipoMetodo.tipo = new Void()}

Tipo ::= Tipo-Primitivo {Tipo.tipo = TipoPrimitivo.tipo}
    | Tipo-Referencia {Tipo.tipo = TipoReferencia.tipo}
    | Tipo-Array {Tipo.tipo = TipoArray.tipo}

Tipo-Primitivo ::= "Bool" {TipoPrimitivo.tipo = new Bool()}
    | "I32" {TipoPrimitivo.tipo = new I32()}
    | "Str" {TipoPrimitivo.tipo = new Str()}
    | "Char" {TipoPrimitivo.tipo = new Char()}

Tipo-Referencia ::= "idClase" {TipoReferencia.tipo = new TipoClase(nombre:idClase.lex)}

Tipo-Array ::= "Array" Tipo-Primitivo {TipoArray.tipo = new Array(tipo: TipoPrimitivo.tipo)}

Lista-Declaración-Variables(1) ::= "idMétodoVariable" {ts.addVariable(new Var(name:
idMetodoVariable.lex, tipo: LDV1.tipo, public: LDV1.public))}
    | "idMétodoVariable" "," Lista-Declaración-Variables(2)  {ts.addVariable(new Var(name:
idMetodoVariable.lex, tipo: LDV1.tipo, public: LDV1.public)); LDV2.tipo = LDV1.tipo;
LDV2.public = LDV1.public;}

Bloque-Método ::= "{" Decl-Var-LocalesN Sentencias "}"
Decl-Var-LocalesN := Decl-Var-Locales Decl-Var-LocalesN | λ
Decl-Var-Locales ::= Tipo ":" Lista-Declaración-Variables ";"
Sentencias ::= Sentencia Sentencias | λ
Sentencia ::= ";"
      | Asignación ";"
      | Sentencia-Simple ";"
      | "if" "(" Expresión ")" Sentencia ElseOp
      | "while" "(" Expresión ")" Sentencia
      | Bloque
      | "return" Expresión ";"
      | "return" ";"
ElseOp ::= "else" Sentencia | λ
Bloque ::= "{" Sentencias "}"
Asignación ::= Asignación-Variable-Simple "=" Expresión
    | Asignación-Self-Simple "=" Expresión
Asignación-Variable-Simple  ::= "idMétodoVariable" "[" Expresión "]"
    | "idMétodoVariable" Encadenado-SimpleN
Encadenado-SimpleN ::= Encadenado-Simple Encadenado-SimpleN | λ
Asignación-Self-Simple ::= "self" Encadenado-SimpleN
Encadenado-Simple ::= "." "idMétodoVariable"
Sentencia-Simple ::= "(" Expresión ")"
Lista-Expresiones ::= Expresión
    | Expresión "," Lista-Expresiones

Expresión ::= ExpOr
ExpOr ::= ExpAnd ExpOrP
ExpOrP ::= "||" ExpAnd ExpOrP | λ
```

```
ExpAnd ::= ExpIgual ExpAndP
ExpAndP ::= "&&" ExpIgual ExpAndP | λ
ExpIgual ::= ExpCompuesta ExpIgualP
ExpIgualP ::= OpIgual ExpCompuesta ExpIgualP | λ
ExpCompuesta ::= ExpAdd OpCompuesto ExpAdd
   | ExpAdd
ExpAdd ::= ExpMul ExpAddP
ExpAddP ::= OpAdd ExpMul ExpAddP | λ
ExpMul ::= ExpUn ExpMulP
ExpMulP ::= OpMul ExpUn ExpMulP | λ
ExpUn ::= OpUnario ExpUn
   | Operando
OpIgual ::= "=="
   | "!="
OpCompuesto ::= "<"
   | ">"
   | "<="
   | ">="
OpAdd ::= "+"
   | "-"
OpUnario ::= "+"
   | "-"
   | "!"
OpMul ::= "*"
   | "/"
   | "%"

Operando ::= Literal
   | Primario EncadenadoOp
Literal ::= "nil"
   | "true"
   | "false"
   | "intLiteral"
   | "stringLiteral"
   | "charLiteral"
Primario ::= ExpresionParentizada
   | AccesoSelf
   | AccesoVar
   | Llamada-Método
   | Llamada-Método-Estático
   | Llamada-Constructor
ExpresionParentizada ::= "(" Expresión ")" EncadenadoOp
AccesoSelf ::= "self" EncadenadoOp
AccesoVar ::= "idMétodoVariable" EncadenadoOp
   | "idMétodoVariable"  "[" Expresión "]"
Llamada-Método ::= "idMétodoVariable" Argumentos-Actuales EncadenadoOp
Llamada-Método-Estático ::= "idClase" "." Llamada-Método EncadenadoOp
Llamada-Constructor ::= "new" "idClase" Argumentos-Actuales EncadenadoOp
   | "new" Tipo-Primitivo "[" Expresión "]"
Argumentos-Actuales ::= "(" ")"
   | "(" Lista-Expresiones ")"
```

```
Encadenado ::= "." Llamada-Método-Encadenado
    | "." Acceso-Variable-Encadenado

Llamada-Método-Encadenado ::= "idMétodoVariable" Argumentos-Actuales EncadenadoOp
Acceso-Variable-Encadenado ::= "idMétodoVariable" EncadenadoOp
    | "idMétodoVariable" "[" Expresión "]"
EncadenadoOp ::= Encadenado | λ
```

2.4 Chequeos semánticos

En la sección 2.2 ya se explicaron brevemente los tipos de error que pueden ocurrir durante el análisis semántico de declaraciones. En esta sección se explicará con mayor detalle cada validación.

Los chequeos semánticos de declaraciones ocurren en dos momentos: durante la construcción de la tabla de símbolos y durante la consolidación de la tabla.

Durante la construcción de la tabla de símbolos se realizan los siguientes chequeos:

1. Al guardar una **clase** en la TS, se valida que no se haya guardado previamente una clase con el mismo nombre.
2. Al guardar una **clase** en la TS, se valida que no herede de una clase base o predefinida de TinyRust+.
3. Al guardar el **constructor** de una clase en la TS, se valida que no haya guardado previamente otro constructor para esa misma clase, dado que TinyRust+ no soporta múltiples constructores por clase.
4. Al guardar un **atributo** de una clase en la TS, se valida que no se haya guardado previamente otro atributo en esa misma clase con el mismo nombre.
5. Al guardar un **atributo** de una clase en la TS, se valida que su nombre no sea “self”, ya que colisionará con la referencia a la instancia de la clase.
6. Al guardar un **método** de una clase en la TS, se valida que no se haya guardado previamente otro método en esa misma clase con el mismo nombre.
7. Al guardar un **método** de una clase en la TS, se valida que su nombre no sea “self”, ya que colisionará con la referencia a la instancia de la clase.
8. Al guardar un **método** de una clase en la TS, se valida que su nombre no sea “create”, ya que en este caso debería declararse el método con la sintaxis del constructor (que no incluye el tipo de retorno ni la palabra reservada “fn”).
9. Al guardar un **argumento** de un método de una clase en la TS, se valida que no se haya guardado previamente otro argumento en ese mismo método con el mismo nombre.
10. Al guardar un **argumento** de una clase en la TS, se valida que su nombre no sea “self”, ya que colisionará con la referencia a la instancia de la clase.

Durante la etapa de consolidación de la tabla de símbolos se realizan los siguientes chequeos semánticos:

1. Para cada **clase** guardada en la TS se valida que su superclase esté definida en la tabla

(a excepción de las clases Object y main, que no poseen superclase).

2. Para cada **clase** guardada en la TS se valida que no posea herencia circular, es decir, que no herede directa o indirectamente de sí misma.
3. Para cada **atributo** de una superclase, se valida que la subclase no defina un atributo con el mismo nombre, ya que TinyRust+ no soporta redefinición de atributos de ningún tipo.
4. Para cada **método estático** de una superclase, se valida que la subclase no defina un método con el mismo nombre, ya que TinyRust+ no soporta redefinición de métodos estáticos.
5. Para cada **método no estático** de una superclase, se valida que la subclase no defina un método con el mismo nombre y distinta firma, ya que TinyRust+ no soporta redefinición de métodos con distinta firma. En este caso se valida que coincidan el tipo de retorno y la cantidad, orden y tipo de argumentos formales para que el método se considere correctamente redefinido.

3. Resultados

3.1 Estructura general

El repositorio del proyecto se encuentra estructurado en una jerarquía de directorios para facilitar su legibilidad. En la Figura 1 puede apreciarse esta jerarquía.

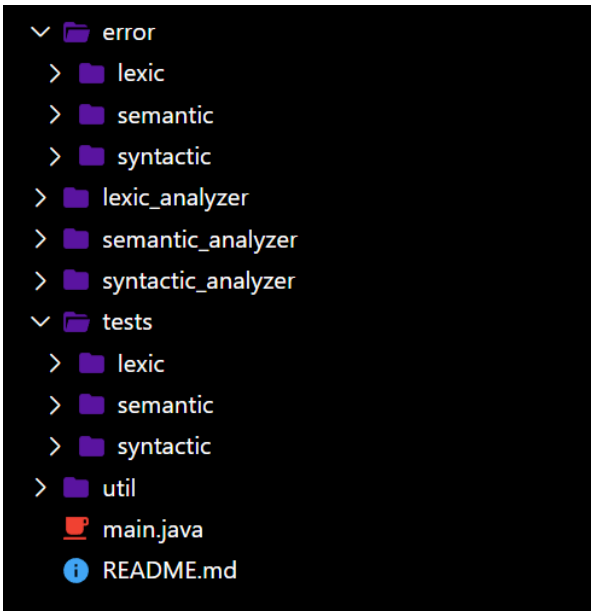


Figura 1. Jerarquía de carpetas

En el directorio raíz se encuentran los siguientes archivos y directorios:

1. Archivo *main.java*: contiene la clase TinyRustCompiler, el punto de entrada del compilador.

2. Carpeta *util*: contiene clases que proveen algún tipo de asistencia mediante métodos. Por ejemplo: la clase Logger (provee métodos necesarios para escribir mensajes en el archivo de salida) y la clase Json (provee métodos para generación de archivos en

formato JSON[2]).

3. Carpeta *tests*: contiene los archivos *.rs* de prueba utilizados para probar el compilador. Contiene una carpeta por cada etapa de análisis con archivos de prueba para esa etapa. Cada archivo de prueba incluye el output esperado al analizarlo.

4. Carpeta *lexic_analyzer:* contiene todas las clases relacionadas al analizador léxico del compilador. Incluye al analizador léxico, un ejecutor y las clases *Token* y *ReservedWords*.

5. Carpeta *syntactic_analyzer*: contiene todas las clases relacionadas al analizador sintáctico del compilador. Incluye al analizador sintáctico y un ejecutor.

6. Carpeta *semantic_analyzer*: incluye una subcarpeta conteniendo todas las clases relacionadas a la tabla de símbolos. Esto incluye clases para cada tipo de entidad de la tabla, cada tipo de dato de TinyRust+ y la clase que representa a la tabla de símbolos propiamente.

7. Carpeta *error*: contiene una clase para cada posible error que pueda ser arrojado al momento de ejecutar el compilador de TinyRust+. Contiene una carpeta por cada etapa de análisis, donde se encuentra una clase para cada error que pueda aparecer en esa etapa.

3.2 **Implementación**

En esta sección se ahondará en la implementación del analizador sintáctico de TinyRust+, explicando el rol, funcionamiento y métodos de cada una de las clases que lo componen. La Figura 2 muestra el diagrama de clases UML de los analizadores léxico y semántico de TinyRust+. La Figura 3 muestra el diagrama de clases UML de la tabla de símbolos de TinyRust+. La división en dos diagramas es meramente para una mejor visualización, dado que contienen una gran cantidad de clases, pero ambos diagramas pertenecen a distintas partes del mismo proyecto. De hecho las clases SyntacticAnalyzer y Logger aparecen en ambos diagramas, se podría decir que son el punto de conexión entre estos.

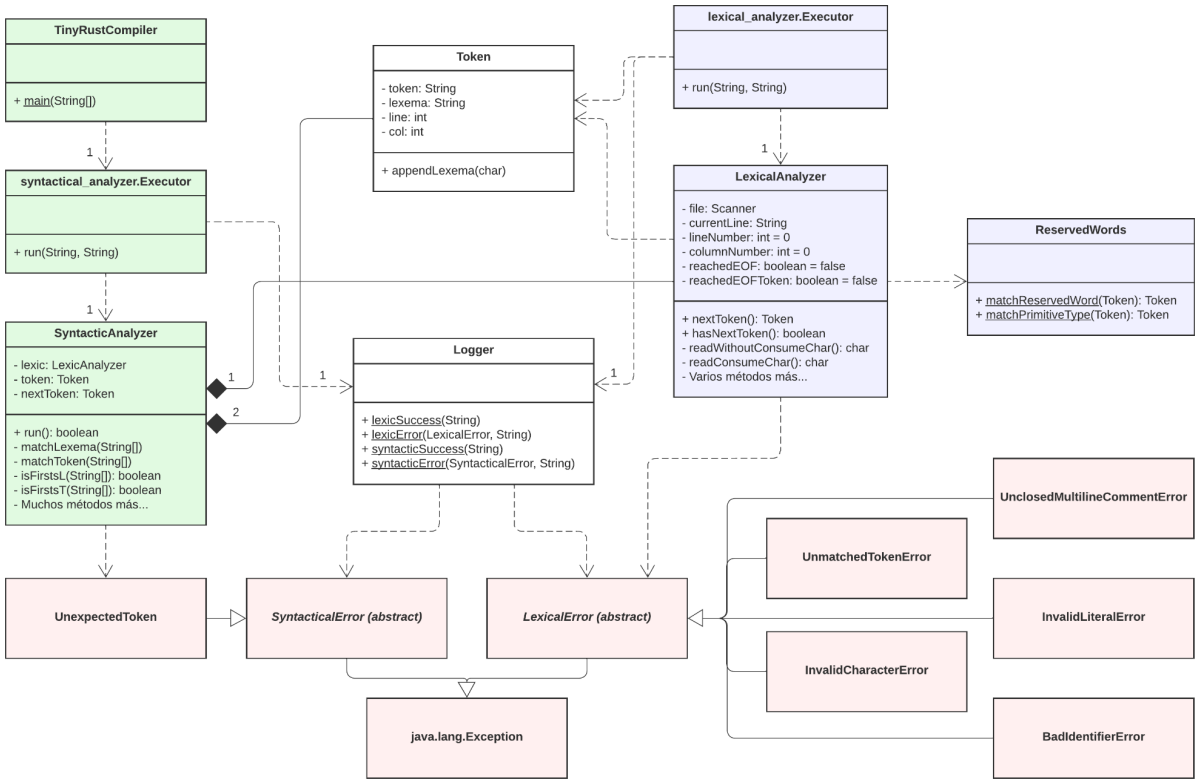


Figura 2. Diagrama de clases UML de los analizadores léxico y semántico de TinyRust+.

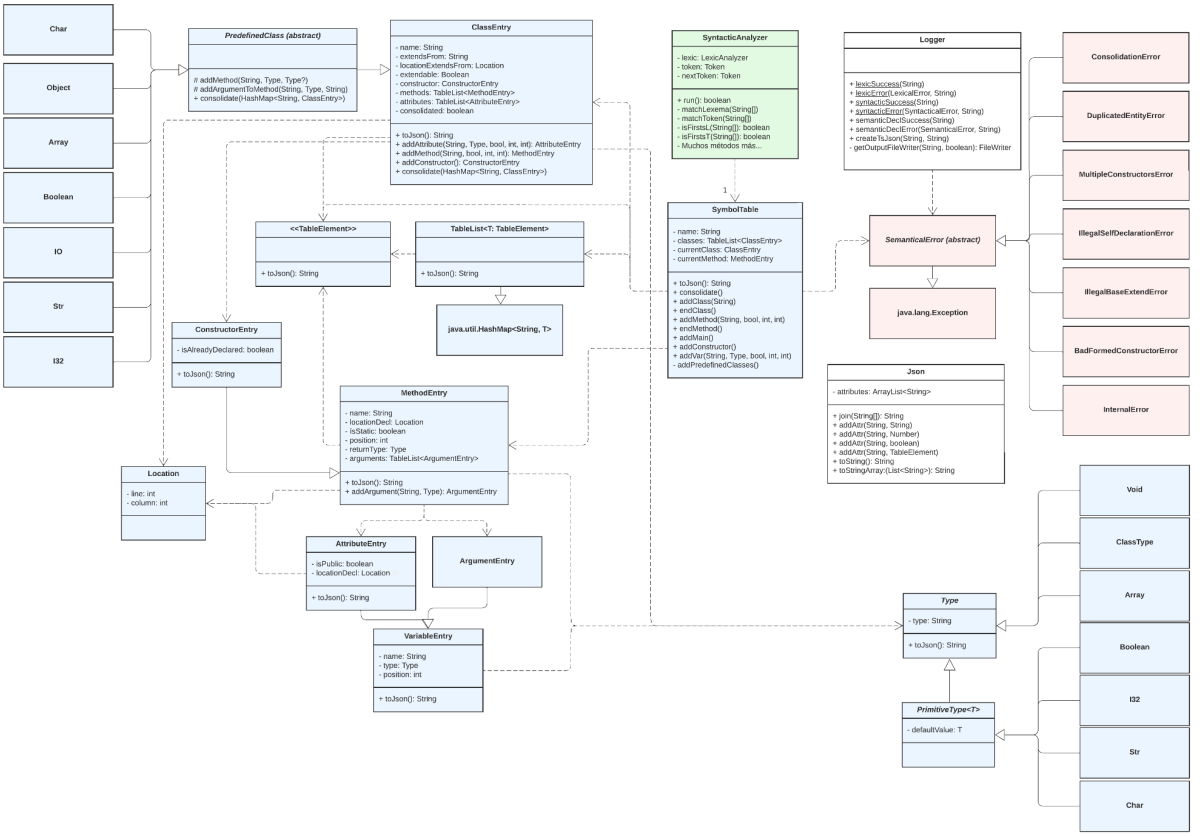


Figura 3. Diagrama de clases UML de la tabla de símbolos de TinyRust+.

3.2.1 **TinyRustCompiler**

Esta es la clase raíz del compilador. Recibe los argumentos de la línea de comandos para los archivos de entrada y salida y se encarga de guardarlos. Además, en el método main de esta clase se instancia la clase *Executor* y se invoca su método *run* para comenzar la ejecución del compilador.

3.2.2 **LexicAnalizer**

Esta clase es la encargada de todo el proceso de análisis léxico. En su constructor se abre el archivo de entrada e inicializan contadores de línea y columna.

LexicAnalizer cuenta con diversos métodos que permiten el correcto análisis léxico de un archivo de texto. Provee únicamente dos métodos públicos, que se describen a continuación:

nextToken: consume parte del archivo de entrada para reconocer y retornar el siguiente token. En caso de haber consumido todo el archivo, devuelve el token EOF (end of file). Si ocurriera algún error al intentar decodificar el archivo, este método arrojaría un error del tipo *LexicalError.*

hasNextToken: verifica si ya se ha alcanzado el final del archivo y devuelve un valor booleano acorde. Este método puede invocarse como validación antes de invocar a *nextToken*. Siempre que *hasNextToken* devuelva *false*, está garantizado que *nextToken* devolverá el token EOF (end of file).

A su vez, *LexicAnalyzer* implementa una gran cantidad de métodos privados que son utilizados directa o indirectamente por el método *nextToken*. Los métodos privados de la clase se listan y describen a continuación.

Validadores: *isValidChar, isLowercaseChar, isUppercaseChar, isAlphabet, isDigit, isOperator, isBlankSpace, isLineBreak.* Estos métodos reciben una variable de tipo char y verifican si ese caracter pertenece a cierto conjunto en particular.

Matchers: *matchIntLiteral*, *matchStringLiteral, matchCharLiteral, matchTypeIdentifier, matchMultilineComment, matchComment.* Estos métodos reciben una variable de tipo char y una variable de tipo *Token*. A partir del caracter inicial recibido, verifican si lo que le sigue en el archivo de entrada coincide con lo que se espera para un token en particular. Devuelven un booleano indicando si se pudo construir por completo el token esperado y actualizan el contenido del token recibido como parámetro. Pueden arrojar excepciones en algunos casos de token mal formado o caracteres inválidos.

consumeSpaces: se encarga de descartar espacios en blanco del archivo de entrada. Arroja *InvalidCharacterError* si se encuentra con algún caracter no perteneciente al alfabeto de entrada

readConsumeChar: lee el siguiente caracter del archivo, lo consume y aumenta el contador de columna. Es decir, una vez leído no hay vuelta atrás. Si se encuentra con el fin de una línea, verifica si hay una línea más. De ser así, incrementa el contador de

línea, reinicia el contador de columna y continúa leyendo la siguiente línea del archivo. Si encuentra el final del archivo, marca que se llegó al final de la lectura y cierra el archivo.

readWithoutConsumeChar: análogo a *readConsumeChar*, lee el siguiente caracter del archivo pero no lo consume. Es decir, no se pierde el caracter leído. Si no se invoca a la función *readConsumeChar*, la función *readWithoutConsumeChar* devuelve siempre el mismo caracter en llamados sucesivos.

3.2.3 **Executor (syntactic_analyzer/Executor)**

Esta clase consta de un único método llamado *run*. Al invocar este método, se crea una instancia de la clase SyntacticAnalyzer que recibe como parámetro la ruta del archivo de entrada. A su vez la ruta de entrada se utiliza para construir el nombre de la ruta de los archivos de salida cambiando únicamente la extensión del archivo, pero conservando el nombre.

Una vez instanciado el analizador, el ejecutor llamará al método *run* del SyntacticcAnalizer, el cual realiza el análisis sintáctico y construye la tabla de símbolos. Este método retorna la tabla de símbolos ya unificada. En caso de un resultado positivo, un mensaje de éxito es escrito en un archivo de salida con extensión *.j* y una representación en formato JSON de la tabla de símbolos se escribe en un archivo con extensión *.ts.json*. Si el archivo no fuera sintácticamente correcto, esta clase intercepta la excepción lanzada por *SyntacticAnalyzer* y en la salida muestra línea, columna y token donde se presentó el error, incluyendo en la descripción si se trata de un error léxico o sintáctico. Si el archivo contuviera declaraciones inválidas semánticamente para TinyRust+, el método run también lanzará una excepción de tipo *SemanticalError*, que será interceptada y escrita en el medio de salida, incluyendo línea, columna y descripción del error.

3.2.4 **SyntacticAnalyzer**

Esta clase es la encargada de todo el proceso de análisis sintáctico. El análisis se hace de forma descendente y predictiva, mientras que la implementación en esta clase es recursiva.

En su constructor se inicializa una instancia de LexicAnalyzer pasándole la ruta del archivo de entrada.

SyntacticAnalyzer provee un único método público, *run*. Este método inicializa la tabla de símbolos y comienza el proceso de análisis sintáctico del archivo utilizando la gramática de TinyRust+. Durante el análisis sintáctico, se irán solicitando y consumiendo tokens del analizador léxico y se verificará que el archivo de entrada sea derivable a partir del símbolo inicial de la gramática, llamando recursivamente métodos de la clase que implementan cada regla de la gramática.

La clase tiene dos atributos privados relevantes: *token* y *nextToken*. Ambos son instancias de la clase Token y son generados por el analizador léxico. Ambas variables

son inicializadas con los primeros dos tokens del archivo al ejecutar el método *run.* Luego su valor va cambiando a medida que el análisis sintáctico avanza y los tokens son *matcheados* en reglas de la gramática. Cada vez que se *matchea* y consume un token, se mueve el valor de *nextToken* a *token* y se solicita un nuevo token al analizador léxico para actualizar el nextToken. En caso de que el analizador léxico arroje una excepción, el analizador sintáctico no la atrapa, sino que la propaga hacia arriba al siguiente invocador.

A su vez, SyntacticAnalyzer posee una enorme cantidad de métodos privados, algunos de ellos se describen a continuación:

matchToken: compara el tipo del token actual contra la lista de tipos de token recibida. Si coincide con algún elemento de la lista, consume el token. Caso contrario, lanza una excepción de tipo *UnexpectedToken*.

matchLexema: compara el lexema del token actual contra la lista de lexemas recibida. Si coincide con algún elemento de la lista, consume el token. Caso contrario, lanza una excepción de tipo *UnexpectedToken*.

isFirstT: compara el tipo del token actual contra la lista de tipos de token recibida y retorna un booleano indicando si coincide con algún elemento de la lista. La principal diferencia con *matchToken* es que *isFirstT* lee el token sin consumirlo.

isFirstL: compara el lexema del token actual contra la lista de lexemas recibida y retorna un booleano indicando si coincide con algún elemento de la lista. La principal diferencia con *matchLexema* es que *isFirstL* lee el token sin consumirlo.

Otros métodos: el resto de métodos privados de la clase *SyntacticAnalyzer* se corresponden cada uno con la regla homónima de la gramática final de TinyRust+. Cada método implementa la lógica de la regla correspondiente. El método ejecutará por completo sin problemas si la secuencia de tokens encontrados es derivable mediante esa regla de la gramática, caso contrario arrojará una excepción sin completar su ejecución.

3.2.5 **SymbolTable**

Esta clase implementa las responsabilidades de la tabla de símbolos. En sus atributos guarda registro del nombre del archivo de entrada, la lista de clases definidas, y la clase y el método analizados actualmente.

Para inicializar la clase únicamente se debe invocar su constructor proporcionando el nombre del archivo. En su lista de clases incluirá por defecto todas las clases predefinidas de TinyRust+.

Posee dos métodos relevantes que son: *consolidate* y *toJson*.

consolidate: es el método que realiza el proceso de consolidación de la tabla una vez que ya se ha consumido todo el archivo de entrada. El método se invoca sin

parámetros y no retorna ningún valor. En este procedimiento se revisa una a una cada clase declarada y se realizan dos pasos. En una primera instancia, se valida que la clase herede de una clase existente (posiblemente Object) y que no exista herencia circular. Si no pasa la validación, el método lanza una excepción. En caso de pasar esta primera validación, se llama al método *consolidate* de la clase pasándole su lista de ancestros para que se agreguen los atributos y métodos heredados a la clase. Luego de consolidar una clase, se marca un flag de "consolidada" de modo que si otra clase hereda de esta, no se realice de nuevo el proceso de consolidación de esta clase hacia sus ancestros. Se realiza una optimización almacenando las relaciones de herencia en un árbol de ancestros. Cuando una clase se agrega al árbol, está garantizado que está consolidada y no tiene herencia cíclica. De modo que al ser referenciada posteriormente, si ya está en el árbol, no se repite todo el proceso de validación y se usa directamente.

toJson: se invoca sin parámetros y devuelve un *String* conteniendo la representación en formato JSON de la tabla y su contenido. Debe invocarse cuando la tabla ya ha sido construida por completo y consolidada. Caso contrario, la información estará incompleta. El JSON se construye de forma recursiva al invocar el método homónimo en los distintos atributos de la tabla.

La clase posee también una serie de métodos para agregar distintos tipos de entidades a la tabla de símbolos:

addClass: recibe un identificador y agrega a la lista de clases declaradas una nueva entrada de clase con ese identificador. Actualiza la referencia a la clase actual apuntando a esta nueva clase, de modo que al agregar métodos, constructores y atributos posteriormente, se agreguen en esta clase.

addMethod: recibe un identificador y un booleano indicando si el método es de clase o instancia para agregar una nueva entrada a la lista de métodos de la clase analizada actualmente, establecida previamente en el método *addClass*.

addMain: agrega una "clase fantasma" a la lista de clases de la tabla. Esta clase fantasma representa al método main. No es heredable y no posee métodos, atributos ni constructor.

addConstructor: inicializa un constructor en la clase analizada actualmente y registra al constructor como el método analizado actualmente. A medida que el análisis avance, se agregarán los parámetros formales al constructor.

addVar: recibe un identificador, un tipo de dato de TinyRust+ y un booleano indicando si la variable es pública. Si actualmente se está analizando una clase pero no un método, esto indica que la variable es un atributo de la clase, por lo cual se llama al método *addAttribute* de la clase analizada. En caso de que se estuviera analizando un método, la variable no se registra, ya que indicaría que es una variable declarada dentro del bloque del método y no es una entidad relevante en esta etapa de análisis.

Finalmente, la clase también contiene setters, getters y algunas funciones extra de control para facilitar su uso, pero que no tienen relevancia para este informe.

3.2.6 **ClassEntry**

La clase *ClassEntry* representa una clase declarada en un archivo de TinyRust+. Entre sus atributos guarda: nombre de la clase, nombre de la superclase, referencia al constructor, booleano indicando si es heredable, lista de atributos y lista de métodos. A su vez posee un flag booleano indicando si la clase ya ha sido consolidada. Al ser una entidad de la tabla de símbolos, implementa el método *toJson*.

Posee métodos para agregar un nuevo atributo, un nuevo método, modificar el constructor y el tipo de retorno y setters/getters para acceder y modificar los distintos atributos de la instancia. Cada uno de estos métodos puede arrojar excepciones si no se respeta la semántica de TinyRust+, por ejemplo, si se intenta agregar dos métodos o dos atributos con el mismo identificador, si se intenta agregar un método o atributo con el identificador “self”, si se declaró más de un constructor para la clase de TinyRust+ o si se declaró el constructor con sintaxis de método común.

El método más relevante de esta clase es *consolidate*, que realiza la consolidación de la clase de TinyRust+ al ser invocado. Este método es invocado únicamente por *SymbolTable* u otra instancia de *ClassEntry* durante la consolidación y recibe el árbol de ancestros completo (ya que previo a la consolidación sólo se conoce el identificador de la superclase, pero no se tiene una referencia concreta). En caso de que la clase no haya sido previamente consolidada y herede de una superclase (la clase *Object* y la clase fantasma *main* heredan de *null*, por lo que la validación es necesaria), se procede a realizar el proceso de consolidación. Como se mencionó anteriormente, *ClassEntry* posee un atributo booleano que indica si la clase ya fue consolidada, de este modo se evita realizar procesamiento innecesario al pasar por el proceso de consolidación más de una vez para la misma clase.

El proceso de consolidación comienza invocando al método *consolidate* de la superclase recursivamente, pasándole el árbol de ancestros. Cuando la superclase ya está consolidada, se realiza la consolidación de atributos y métodos de la clase actual. Cada atributo de la superclase se agrega a la lista de atributos de la subclase solo si esta no posee un atributo con el mismo identificador. En caso de colisión, se lanza una excepción, dado que no se permite redefinir atributos en TinyRust+. Los métodos también se heredan directamente si no hay colisión de identificadores. En caso de que la subclase redefina un método, el método de la superclase **no** se agrega a la subclase y se valida que su firma coincida. En esta validación se comprueba que el método no sea estático y que coincidan el tipo de retorno, la cantidad de argumentos formales y tipo y orden de los argumentos. En caso de no pasar la validación, se lanza una excepción de tipo *ConsolidationError* con un mensaje explicativo al usuario. Tras consolidar métodos y atributos exitosamente, se actualiza a *true* el booleano que indica que la clase ya ha sido consolidada y finaliza la ejecución del método.

3.2.7 MethodEntry

Esta clase representa un método de una clase definida en TinyRust+. Incluye distintos atributos que registran el nombre del método, su posición, tipo de retorno, lista de argumentos y un booleano indicando si es un método de instancia o de clase.

Al ser una entidad de la tabla de símbolos, implementa el método *toJson*, que genera la representación JSON de únicamente el método. Posee una serie de setters y getters para acceder y modificar sus atributos. Además de estos, posee el método *addArgument* para agregar un nuevo argumento a la lista de argumentos de la instancia. Para agregar un argumento se debe invocar al método con el identificador y tipo del nuevo argumento, que puede arrojar una excepción si el identificador es “self” o ya existe otro argumento con el mismo identificador.

3.2.8 ConstructorEntry

La clase *ConstructorEntry* hereda de *MethodEntry* y representa el constructor de una clase de TinyRust+. Dado que la lógica genérica de todo método está implementada en la superclase *MethodEntry*, esta clase sólo implementa una versión simplificada del método *toJson* para el constructor y se guarda registro de si el constructor fue declarado de forma explícita o es el constructor implícito por defecto. Este valor se consulta a la hora de validar si una clase de TinyRust+ posee más de un constructor declarado.

3.2.9 VariableEntry

Esta clase representa una variable del lenguaje TinyRust+. En sus atributos guarda el nombre, tipo y posición de la variable. Implementa getters y el método *toJson*. No es una entidad utilizada explícitamente en la tabla de símbolos, pero *AttributeEntry* y *ArgumentEntry* heredan de esta clase.

3.2.10 AttributeEntry

AttributeEntry representa una variable del lenguaje TinyRust+ y hereda de la clase *VariableEntry*. Únicamente agrega el registro de si el atributo es público o privado, dado que el tipo, valor y posición los guarda la superclase. Implementa el método *toJson* incluyendo todos estos datos en la representación JSON.

3.2.11 ArgumentEntry

AttributeEntry representa un argumento formal de un método en el lenguaje TinyRust+ y hereda de la clase *VariableEntry*. No agrega ninguna funcionalidad a la superclase, su valor es meramente semántico.

3.2.12 T**ipos Predefinidos**

La clase abstracta *PredefinedClass* representa una clase predefinida de TinyRust+. Extiende de *ClassEntry* y es a su vez heredada por las clases *Array, Bool, Char, I32, IO*,

Object y *Str*. Todas estas clases concretas representan cada una a una clase predefinida distinta de TinyRust+ y son agregadas como entidad a la lista de clases de la tabla de símbolos durante su inicialización. *PredefinedClass* redefine el método *consolidate* con el tope recursivo. Ya que todas las clases predefinidas heredan de *Object* (o es la clase *Object* propiamente), no se modifican al consolidarse, por lo cual se consideran consolidadas por defecto. Es decir, el método *consolidate* de esta clase únicamente actualiza a true el booleano indicando que la clase ya está consolidada y retorna luego de esto. Todas las clases predefinidas se inicializan con su constructor en *null* y como no heredables.

Cada clase que hereda de *PredefinedClass* debe llamar al constructor de *PredefinedClass* con el identificador del tipo (es decir, el nombre de la clase en TinyRust+). Cada una de estas clases puede llamar en su constructor a los métodos de la superclase para registrar la serie de métodos que la clase predefinida de TinyRust+ implementa. Por ejemplo, la clase *Str* registra los métodos *length, concat y substr*; las clases *Array* e *IO* también implementan sus propios métodos.

3.2.13 Tipos de Variable

Se implementó la clase abstracta *Type* para representar a los distintos tipos de datos de TinyRust+ a la hora de declarar el tipo de una variable, argumento, atributo o tipo de retorno de un método. Esta clase únicamente posee un atributo que registra el identificador del tipo de dato de TinyRust+. Por ejemplo, “Str”, “Char”, “I32”, etc. Posee un getter para este atributo e implementa el método *toJson*.

Las clases *Array, PrimitiveType, Void* y *ClassType* heredan de *Type*. El identificador de la clase *Array* incluye también el tipo de dato contenido en el arreglo (por ejemplo, “Array-Str” o “Array-I32”). El tipo *ClassType* representa a cualquier clase de TinyRust+ definida por el usuario, simplemente guarda el identificador de la clase, pero mantiene la interfaz consistente con *Type*, a diferencia de *ClassEntry*.

De la clase abstracta *PrimitiveType* extienden *Bool, Char, I32* y *Str,* los cuatro tipos primitivos de TinyRust+. Lo único que agrega esta clase intermedia es la posibilidad de especificar el valor por defecto de las variables de este tipo. Es una funcionalidad que no se utiliza por el momento, pero que fue pensada para la siguiente etapa de desarrollo del compilador.

3.2.14 Otras clases e interfaces

Para la implementación de la tabla de símbolos de TinyRust+ se crearon también algunas clases e interfaces que no se corresponden directamente con ninguna entidad de TinyRust+, pero que facilitan la extensibilidad y legibilidad del código.

Por ejemplo, la interfaz *TableElement* es implementada por todos las entidades de la tabla de símbolos y exige que su clase implemente el método *toJson*. El método *toJson* permite construir de manera recursiva la representación en formato JSON de la tabla de

símbolos, ya que cada elemento llama al método *toJson* de sus atributos para armar su propia representación.

La clase genérica *TableList* hereda de *HashMap* y sirve para almacenar conjuntos de entidades del mismo tipo de la tabla de símbolos, por ejemplo: lista de métodos, atributos o clases. A su vez, esta clase agrega funcionalidad a *HashMap* ya que implementa también el método *toJson*.

La clase Location simplemente guarda un número de línea y columna del archivo de entrada donde se encontró un Token, usada para mostrar correctamente la información en los errores semánticos.

3.3 Errores

Para el manejo de errores de cada etapa de análisis se creó una clase abstracta distinta que hereda de la clase Exception de Java. Por ejemplo, los errores lanzados por el analizador léxico heredan de la clase abstracta *LexicalError*, mientras que los errores del analizador sintáctico heredan de *SyntacticalError* y los errores semánticos heredan de *SemanticalError*. Una nueva instancia de cualquiera de estas clases de excepción espera recibir el número de línea y columna donde ocurrió el error junto con una descripción. A la hora de mostrar el mensaje de error, este incluirá estos tres datos.

En el caso del análisis léxico, para cada tipo de error distinto que pueda ocurrir durante el análisis se creó una clase distinta que herede de *LexicalError*. Los distintos tipos de error posibles en esta etapa se explican en la sección 2.2.

En el caso del análisis sintáctico, todos los errores lanzados son de tipo *UnexpectedToken*, que hereda de la clase *SyntacticalError*. Esta excepción se produce cuando se encuentra un token que no coincide con ninguna regla gramatical de TinyRust+, teniendo en cuenta la secuencia previa de tokens. Este error incluye en cada caso un mensaje especificando el lexema que se encontró y el o los posibles tokens/lexemas que se esperaba encontrar en ese contexto. En ocasiones este error incluye una descripción de la estructura gramatical que se esperaba en vez de incluir la especificación de cada posible token esperado. Por ejemplo, cuando se espera una expresión y se recibe un token que no podría ser el primer token de ninguna expresión válida, el mensaje dirá “se esperaba una expresión, se encontró [token]”. Esto es debido a que un mensaje listando el conjunto de 14 tokens válidos para comenzar una expresión podría no resultar de ayuda para el usuario que intenta compilar un archivo de TinyRust+.

En el caso del análisis semántico de declaraciones, para cada tipo de error distinto que pueda ocurrir durante el análisis se creó una clase distinta que herede de *SemanticalError*. Los distintos tipos de error posibles en esta etapa también se explican en la sección 2.2. Para poder indicar la línea y columna donde ocurrió un error de consolidación, cada entidad de la tabla de símbolos guarda registro de la línea y columna donde fue declarada. Como caso particular, en la implementación se incluye un error de

tipo *InternalError*. Esta es una excepción que indica un error de programación, se utiliza en situaciones donde es prolijo manejar una posible excepción, pero donde teóricamente no es posible que se cumplan las condiciones para lanzar el error. Por ejemplo, si se invocara la función para agregar un argumento al método actual, pero no se estuviera analizando ningún método actualmente.

3.4 Casos de prueba

En el código del proyecto se incluyen los casos de prueba que se utilizaron para poner a prueba el funcionamiento de cada etapa de análisis. Dichos archivos se pueden encontrar en el directorio *tests* del proyecto, donde se encuentra una carpeta por cada etapa de análisis, conteniendo archivos de prueba con casos exitosos y casos de fallo.

La lista de tests consiste en 30 archivos de prueba para el análisis léxico y más de 100 archivos de prueba para el análisis sintáctico. Se detalla a continuación la estructura de los casos de prueba para la etapa de análisis sintáctico.

Particularmente el archivo de test número 0 corresponde al ejemplo de código de TinyRust+ provisto por la cátedra en el manual del lenguaje. El análisis sintáctico de este archivo es exitoso.

Los primeros cuatro archivos de prueba, sumados a unos pocos archivos extra entre los 100 casos de prueba, contienen casos exitosos. Estos archivos contienen una gran cantidad de código sintácticamente válido de TinyRust+ para poner a prueba que casos de uso comunes y casos borde sean reconocidos correctamente.

Mientras que el resto de archivos de prueba (la gran mayoría) son casos de falla, en los que se espera que el compilador arroje cierto error en un token en particular que no respete la gramática del lenguaje. Cada archivo incluye al final un comentario especificando en qué lugar del archivo se encontró un token inesperado y cuál es el o los tokens esperados en ese lugar del archivo, dado el contenido previo.

3.5 Compilado y ejecución

Para compilar el proyecto, deben ejecutarse los siguientes comandos desde el directorio raíz del proyecto.

Paso 1: compilar.

```
javac lexic_analyzer/*.java syntactic_analyzer/*.java semantic_analyzer/symbol_table/*.java util/*.java *.java
```

Paso 2: generar el jar.

```
jar cfm etapa3.jar Manifest.txt *
```

Para ejecutar el proyecto debe ejecutarse el siguiente comando:

```
java -jar etapa3.jar <ARCHIVO_FUENTE>
```

Es obligatorio especificar la ruta del archivo fuente que TinyRust+ analizará. En el directorio *tests* se pueden hallar archivos de prueba para alimentar el comando. Por ejemplo:

```
java -jar etapa3.jar ./tests/semantic/"test (2).rs"
```

El comando generará entre uno y dos archivos de salida. Siempre generará un archivo con extensión .j e igual nombre que el archivo de entrada en el cual se registra el resultado del análisis. Este resultado puede ser un mensaje de éxito o la información de un error, en caso de que el análisis no se haya podido completar correctamente. Solo en caso de que el análisis haya sido exitoso, se creará un segundo archivo de igual nombre y extensión .ts.json en el cual se encuentra una representación en formato JSON de la tabla de símbolos con todo su contenido.

4. Conclusiones

Tras finalizar la tercera etapa del desarrollo del compilador de TinyRust+, el proyecto ya se percibe más sólido. Los archivos de prueba se ven cada vez más cercanos al código real y los requerimientos del manual que aún no se cumplen son pocos. El analizador sintáctico implementado en la etapa previa permitió agregar fácilmente las acciones semánticas para interactuar con la tabla de símbolos. A diferencia de la etapa anterior, el mayor desafío en esta etapa fue el diseño de la arquitectura y la representación de cada entidad del sistema.

5. Referencias

1. ASCII-CODE. ASCII table according to Windows-1252. https://www.ascii-code.com/

2. JSON. ECMA-404 The JSON Data Interchange Standard. https://www.json.org/json-es.html